



Universidad Mariano Gálvez de Guatemala

CENTRO UNIVERSITARIO EL NARANJO, MIXCO

FACULTAD DE INGENIERÍA EN SISTEMAS Y CIENCIAS DE LA
COMPUTACIÓN

SEGURIDAD Y AUDITORÍA DE SISTEMAS

Tema

Sistema de Autenticación

Integrantes:

Diego Antonio Beteta García 9490-22-12878

Mario Roberto Rompich Yoc 9490-17-17052

Jonathan Rogelio Herrera Soto 9490-22-11551

Guatemala, 10 de septiembre de 2026

Introducción

La autenticación es uno de los puntos más importantes de cualquier sistema que permita crear cuentas y acceder a información privada. En la práctica, muchas aplicaciones comienzan con una combinación de usuario y contraseña porque es un mecanismo conocido y fácil de implementar. El problema aparece cuando una contraseña se filtra, se reutiliza en diferentes servicios o es obtenida mediante ingeniería social. En ese escenario, depender únicamente de la contraseña significa que una sola credencial puede ser suficiente para comprometer una cuenta.

A partir de esta situación, el proyecto de curso plantea fortalecer el modelo de autenticación incorporando dos métodos de segundo factor: TOTP y WebAuthn/passkeys. La elección tiene una razón práctica. TOTP es una opción conocida, accesible y compatible con aplicaciones de autenticación, mientras que WebAuthn/passkeys permite avanzar hacia una autenticación resistente al phishing. NIST señala que los OTP no son resistentes al phishing porque el código se introduce manualmente y puede ser retransmitido a un sitio impostor, mientras que WebAuthn puede proporcionar resistencia al phishing mediante la vinculación con el identificador del sitio autenticado (National Institute of Standards and Technology [NIST], 2025).

El objetivo de este documento es justificar la selección, explicar cómo funcionaría cada método dentro del proyecto y establecer una propuesta de implementación que sea realista para un trabajo universitario. También se pretende dejar claro qué problema resuelve cada mecanismo, cuáles son sus limitaciones y cómo se podría comprobar mediante indicadores si la solución mejora la seguridad y la experiencia de acceso.

Contexto del proyecto y necesidad de un segundo factor

De acuerdo con las decisiones técnicas planteadas para el proyecto, la sesión de usuario se manejará mediante cookies de sesión del servidor. También se contempla el uso de Laravel como parte de la solución de autenticación. Dentro de este escenario, agregar un segundo factor permite que la contraseña deje de ser la única barrera de acceso. La idea es que, después de comprobar la contraseña, el sistema solicite una evidencia adicional de que la persona que intenta entrar tiene control de un dispositivo, credencial o mecanismo previamente registrado.

Es importante diferenciar autenticación de sesión. Una vez que el usuario completa correctamente el proceso de autenticación, la aplicación necesita mantener la sesión de manera segura para no pedir las credenciales en cada solicitud. La cookie de sesión cumple esa función. Sin embargo, la cookie no debe confundirse con el segundo factor. El segundo factor interviene en el proceso de demostrar la identidad; la sesión mantiene el resultado de esa autenticación durante el tiempo permitido.

La propuesta también contempla códigos de recuperación. Estos códigos son importantes porque ningún mecanismo de segundo factor debe diseñarse suponiendo que el usuario nunca perderá su teléfono, dispositivo o credencial. Laravel Fortify contempla el uso de códigos de recuperación asociados a la autenticación de dos factores y permite regenerarlos cuando corresponde (Laravel, s. f.). En el proyecto, estos códigos se consideran principalmente un mecanismo de recuperación y continuidad, no un sustituto equivalente de un segundo factor resistente al phishing.

Objetivo de seguridad

El objetivo principal es reducir la probabilidad de que una contraseña comprometida sea suficiente para obtener acceso a una cuenta. Para conseguirlo, el sistema combinará una credencial principal con una segunda evidencia. La propuesta busca, además, que la segunda capa no genere una experiencia innecesariamente complicada para los usuarios.

Reducir el impacto de una contraseña expuesta.

Ofrecer una opción de segundo factor ampliamente compatible mediante TOTP.

Incorporar una opción más resistente al phishing mediante WebAuthn/passkeys.

Mantener un mecanismo de recuperación mediante códigos de recuperación.

Registrar eventos relevantes de autenticación para facilitar el seguimiento de incidentes.

Principio de defensa en profundidad

La propuesta se basa en defensa en profundidad: si una barrera falla, todavía existe otra capa que dificulta el acceso. Esto no significa que el sistema sea imposible de atacar. Significa que el costo y la dificultad de un acceso no autorizado aumentan. NIST reconoce diferentes tipos de autenticadores y establece requisitos relacionados con resistencia al phishing, resistencia a repetición y protección de las credenciales (NIST, 2025).

Primer método: TOTP como segundo factor

TOTP significa Time-Based One-Time Password, o contraseña de un solo uso basada en tiempo. Su funcionamiento se define en el RFC 6238, que describe una variante de HOTP donde el factor cambiante es el tiempo. En términos sencillos, el usuario registra un secreto compartido con el servicio y una aplicación autenticadora genera códigos temporales a partir de ese secreto y de la hora actual (M'Raihi et al., 2011).

El comportamiento es fácil de entender para un usuario. Después de activar el segundo factor, la aplicación autenticadora muestra un código que cambia periódicamente. Cuando el usuario inicia sesión con su contraseña, el sistema solicita ese código. El servidor verifica el valor esperado y, si coincide dentro de la ventana de tiempo permitida, permite continuar con la autenticación.

Flujo propuesto de TOTP

Paso 1: el usuario inicia sesión con su contraseña.

Paso 2: el sistema detecta que la autenticación de dos factores está activa.

Paso 3: el sistema solicita el código TOTP.

Paso 4: el usuario consulta el código en su aplicación autenticadora.

Paso 5: el usuario introduce el código.

Paso 6: el servidor verifica el código y, si es correcto, completa la autenticación.

El RFC 6238 establece que TOTP se calcula utilizando un valor secreto y un factor temporal. El documento propone por defecto un intervalo de 30 segundos para el paso temporal, aunque una implementación puede establecer parámetros de acuerdo con sus necesidades (M'Raihi et al., 2011). En el proyecto, no conviene ampliar demasiado la ventana de aceptación porque hacerlo aumenta la posibilidad de aceptar códigos fuera del momento esperado.

Ventajas de TOTP

Es un método conocido por los usuarios y compatible con diferentes aplicaciones autenticadoras.

No depende de que el sistema envíe un mensaje de texto en cada inicio de sesión.

Los códigos son temporales y no deben reutilizarse indefinidamente.

Es una alternativa relativamente sencilla para un prototipo universitario.

Laravel puede integrarse con mecanismos de autenticación de dos factores y recuperación.

Limitaciones de TOTP

La principal limitación de TOTP es que no debe presentarse como un mecanismo resistente al phishing. El usuario ve un código y lo escribe en una página. Un atacante que controle un sitio falso puede intentar obtener ese código y retransmitirlo al servicio legítimo. NIST señala expresamente que los OTP ingresados manualmente no se consideran resistentes al phishing (NIST, 2025).

Por esta razón, TOTP se selecciona en el proyecto como una segunda capa práctica y compatible, pero se complementa con WebAuthn/passkeys para los escenarios en los que se busca una protección mayor frente al phishing.

Segundo método: WebAuthn y passkeys

WebAuthn, o Web Authentication, es una API web basada en credenciales de clave pública. La especificación de W3C define un mecanismo mediante el cual las aplicaciones web pueden crear y utilizar credenciales criptográficas vinculadas a un sitio o relying party. En lugar de depender de un secreto que el usuario tenga que escribir, el proceso utiliza una clave privada que permanece bajo el control del autenticador y una clave pública que puede registrar el servicio (World Wide Web Consortium [W3C], 2026).

Las passkeys son una forma de experiencia de usuario construida alrededor de credenciales de clave pública y mecanismos como WebAuthn. El usuario puede desbloquear el autenticador con el mecanismo disponible en su dispositivo, como un PIN o una característica biométrica, sin que la aplicación web tenga que recibir esa información biométrica. Lo

importante para el proyecto es que el servidor verifica una prueba criptográfica de posesión de la credencial.

¿Por qué se eligió WebAuthn/passkeys?

La decisión está directamente relacionada con el problema del phishing. En TOTP, el usuario puede copiar un código desde su aplicación autenticadora y escribirlo en una página falsa. En WebAuthn, la autenticación se vincula criptográficamente con el sitio para el que se creó la credencial. NIST identifica WebAuthn como un ejemplo de autenticación resistente al phishing mediante la vinculación con el nombre del verificador (NIST, 2025).

Esta característica hace que WebAuthn sea un complemento muy útil para el proyecto. La propuesta no busca obligar a todos los usuarios a utilizar una llave física. La experiencia puede aprovechar las credenciales disponibles en el dispositivo del usuario, mientras que una llave de seguridad compatible puede funcionar como otra posibilidad cuando el entorno lo requiera.

Flujo propuesto de passkey

Paso 1: el usuario registra una passkey desde su cuenta.

Paso 2: el navegador solicita al autenticador que cree una credencial.

Paso 3: el autenticador protege la clave privada y entrega al servicio la información pública necesaria.

Paso 4: en un inicio de sesión posterior, el servidor genera un desafío.

Paso 5: el autenticador firma el desafío utilizando la credencial correspondiente.

Paso 6: el servidor verifica la firma y completa la autenticación.

La especificación WebAuthn de W3C describe precisamente este modelo de credenciales de clave pública vinculadas a un relying party. El navegador media el acceso del sitio al

autenticador y el autenticador requiere la participación del usuario según sus propias capacidades (W3C, 2026).

Ventajas y limitaciones

Entre sus ventajas destaca la resistencia al phishing, ya que la prueba criptográfica está vinculada al sitio legítimo. También reduce la necesidad de que el usuario memorice o copie códigos. La principal dificultad para un prototipo universitario es que requiere comprender correctamente el registro de credenciales, los desafíos, las respuestas del autenticador y la relación entre navegador, servidor y dispositivo.

Otra consideración es la recuperación. Si una persona pierde el dispositivo y no tiene otra credencial registrada, el sistema debe proporcionar un camino controlado de recuperación. Por eso, el proyecto mantiene los códigos de recuperación como respaldo y recomienda registrar más de una credencial cuando el caso de uso lo permita.

Comparación de los dos métodos

La selección de TOTP y WebAuthn/passkeys no pretende demostrar que uno sea universalmente mejor que el otro. Los dos resuelven necesidades diferentes. TOTP aporta compatibilidad y una experiencia conocida, mientras que WebAuthn aporta una protección especialmente valiosa contra el phishing.

Aspecto	TOTP	WebAuthn/Passkeys	Propuesta
Interacción	Código temporal	Confirmación con autenticador	Ofrecer ambas opciones
Phishing	No resistente	Resistente cuando se implementa correctamente	Priorizar passkeys
Complejidad	Media-baja	Media	Implementar progresivamente
Recuperación	Códigos de recuperación	Credencial alternativa + recuperación	Mantener respaldo
Dependencia	Aplicación autenticadora	Dispositivo/autenticador	Registrar alternativas
Experiencia	Introducir código	Autenticación directa	Permitir elegir método

Estrategia de integración

La estrategia recomendada para el proyecto es mantener la contraseña como primer factor y ofrecer dos alternativas de segundo factor. TOTP puede funcionar como una opción accesible para usuarios que ya utilizan una aplicación autenticadora. WebAuthn/passkeys puede presentarse como la opción preferida cuando el dispositivo sea compatible, especialmente por su resistencia al phishing.

En el diseño de la interfaz, el usuario debería poder revisar desde su perfil qué métodos tiene activos. Una pantalla de seguridad podría mostrar el estado de TOTP, las passkeys registradas y la posibilidad de generar nuevos códigos de recuperación. Las operaciones

sensibles, como regenerar códigos o eliminar una credencial, deberían exigir una autenticación fuerte para evitar que una sesión comprometida sea suficiente para debilitar la cuenta.

Códigos de recuperación

Los códigos de recuperación cumplen una función importante, pero deben describirse correctamente. No son una tercera tecnología equivalente a TOTP o WebAuthn. Son una alternativa para recuperar el acceso cuando el segundo factor principal no está disponible. Laravel Fortify documenta el manejo de códigos de recuperación para autenticación de dos factores (Laravel, s. f.).

En la propuesta, los códigos deben mostrarse al usuario durante la activación y recomendarse guardar en un lugar seguro. Cada código debe poder utilizarse una sola vez y, después de usarse o regenerarse, debe quedar invalidado. También es importante evitar mostrarlos nuevamente de forma innecesaria.

Proceso de inicio de sesión

El proceso comienza con el ingreso de las credenciales principales. Si la contraseña es correcta y la cuenta tiene segundo factor habilitado, el sistema solicita el método correspondiente. Si el usuario selecciona TOTP, se presenta el formulario del código temporal. Si selecciona una passkey, el navegador inicia el desafío WebAuthn y el usuario confirma mediante su autenticador.

Una vez verificado el segundo factor, el servidor establece o completa la sesión. La cookie de sesión debe configurarse con atributos de seguridad apropiados para reducir riesgos de robo o uso indebido. El proyecto debe utilizar HTTPS en el entorno publicado, porque la

protección del transporte es una condición básica para que las credenciales y sesiones no viajen expuestas.

Controles adicionales

Limitar intentos fallidos de TOTP para reducir ataques de adivinación.

Registrar eventos relevantes de inicio de sesión y cambios de factores.

Permitir cerrar sesiones activas cuando exista sospecha de compromiso.

Solicitar autenticación fuerte antes de modificar los factores de seguridad.

No almacenar contraseñas en texto plano; utilizar mecanismos de hashing apropiados.

Proteger las claves y secretos relacionados con TOTP.

Mantener HTTPS en producción.

NIST establece requisitos relacionados con limitación de intentos, resistencia a repetición y protección de autenticadores. En particular, para OTP se requiere implementar mecanismos efectivos de limitación de intentos, y los protocolos resistentes al phishing deben vincular la autenticación con el verificador legítimo (NIST, 2025).

Riesgos, experiencia del usuario y evaluación

Una propuesta de autenticación debe considerar tanto la seguridad como la experiencia de uso. Un sistema demasiado difícil puede provocar que los usuarios desactiven los controles o recurran a prácticas inseguras. Por eso, el proyecto debe explicar al usuario qué método está utilizando y qué hacer si pierde el acceso a su segundo factor.

Riesgos de TOTP

El riesgo principal es el phishing. Un atacante puede crear una página que solicite el código temporal y tratar de retransmitirlo rápidamente. También existe el riesgo de pérdida del dispositivo que contiene la aplicación autenticadora. El proyecto puede disminuir estos

problemas con limitación de intentos, sesiones cortas en operaciones sensibles, advertencias claras y un mecanismo de recuperación controlado.

Riesgos de WebAuthn/passkeys

WebAuthn reduce de manera importante el riesgo de phishing, pero no elimina todos los riesgos de seguridad de una aplicación. La cuenta puede seguir expuesta si el usuario entrega su contraseña por otros medios, si existe una vulnerabilidad en la aplicación o si un atacante obtiene acceso al dispositivo. También debe planificarse correctamente la recuperación de cuenta y el registro de credenciales adicionales.

Indicadores de evaluación

Para demostrar el beneficio del proyecto, se proponen indicadores que puedan medirse durante las pruebas:

Porcentaje de usuarios que completan correctamente la configuración del segundo factor.

Tiempo promedio para completar un inicio de sesión con TOTP.

Tiempo promedio para completar un inicio de sesión con passkey.

Cantidad de intentos fallidos antes y después de habilitar el segundo factor.

Cantidad de cuentas protegidas con al menos un segundo factor.

Número de eventos de recuperación de cuenta.

Cantidad de intentos de autenticación rechazados.

Estos indicadores permitirían presentar el proyecto con resultados concretos en lugar de limitarse a afirmar que la aplicación es más segura. Por ejemplo, se podría realizar una prueba con un grupo de usuarios y comparar el tiempo de acceso con contraseña solamente frente al acceso protegido con segundo factor. También se podría registrar cuántos usuarios logran completar la configuración sin asistencia.

Criterio de éxito

El proyecto podría considerarse exitoso si el prototipo permite activar, utilizar y desactivar los dos métodos de manera controlada; si las sesiones solo se establecen después de completar el proceso de autenticación requerido; si los códigos de recuperación funcionan como mecanismo de respaldo; y si las pruebas muestran una mejora verificable en la protección de las cuentas sin generar una tasa excesiva de errores de usuario.

Conclusiones

La selección de TOTP y WebAuthn/passkeys responde a dos necesidades complementarias del proyecto. TOTP ofrece una alternativa práctica y conocida para implementar autenticación de dos factores. Su funcionamiento basado en códigos temporales está definido por el RFC 6238 y cuenta con amplia adopción. Sin embargo, su principal limitación es que el código puede ser introducido manualmente en un sitio falso, por lo que no debe considerarse resistente al phishing (M'Raihi et al., 2011; NIST, 2025).

WebAuthn/passkeys aporta una segunda opción con un modelo criptográfico diferente. La credencial se basa en claves públicas y el proceso está vinculado al sitio para el que fue creada. Esto permite ofrecer una defensa mucho más fuerte frente al phishing. La especificación de W3C establece precisamente el uso de credenciales de clave pública para autenticar usuarios en aplicaciones web (W3C, 2026).

En el contexto del proyecto, la combinación de ambos métodos resulta más útil que elegir uno y descartar el otro. TOTP facilita la adopción y permite que usuarios con diferentes dispositivos tengan una alternativa conocida. WebAuthn/passkeys ofrece una ruta de

autenticación más resistente al phishing y representa una oportunidad para incorporar una tecnología moderna de seguridad al prototipo.

Los códigos de recuperación completan la propuesta porque contemplan una situación muy real: la pérdida o indisponibilidad del segundo factor. No deben utilizarse como sustituto permanente de la autenticación fuerte, sino como una salida controlada para recuperar la cuenta y volver a registrar un método de autenticación.

Finalmente, el proyecto no debería evaluarse únicamente por haber implementado dos métodos. Su calidad debe demostrarse mediante pruebas e indicadores. Medir tiempos de acceso, errores, intentos fallidos, activaciones y recuperaciones permitirá explicar con datos si la propuesta es funcional y si la seguridad adicional puede incorporarse sin perjudicar de manera innecesaria la experiencia del usuario.

Referencias

- Laravel. (s. f.). *Laravel Fortify*. <https://laravel.com/docs/fortify>
- M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-based one-time password algorithm (RFC 6238)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6238>
- National Institute of Standards and Technology. (2025). *Digital identity guidelines: Authentication and lifecycle management (NIST SP 800-63B-4)*. U.S. Department of Commerce. <https://pages.nist.gov/800-63-4/sp800-63b.html>
- World Wide Web Consortium. (2026). *Web Authentication: An API for accessing public key credentials—Level 3*. <https://www.w3.org/TR/webauthn/all/>